PPSU
P P SAVANI UNIVERSITY

NAAC ACCREDITED A+ GRADE

UNIT 1: Introduction to Automata

AUTOMATA THEORY AND COMPILER DESIGN (SECE3210)

Prepared by

Khushbu Chauhan

CONTENT

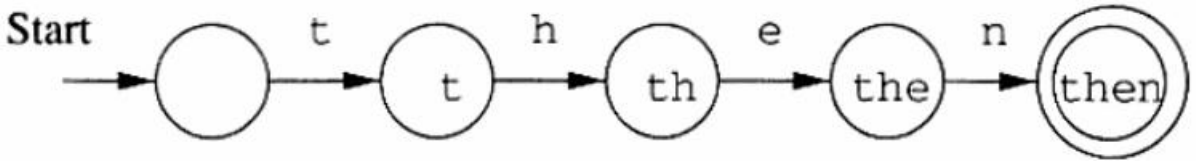
- Languages
- Definitions
- Regular Expressions, Regular Grammars
- Acceptance of Strings and Languages
- Finite Automaton Model
- DFA, NFA
- conversion of NFA to DFA
- Conversion of Regular Expression to NF

WHAT DOES AUTOMATA MEAN?

- It is the plural of automaton, and it means “something that works automatically”.
- Automata theory is the study of abstract computational devices and the computational problems that can be solved using them.
- Abstract devices are (simplified) models of real computations

INTRODUCTION TO AUTOMATA

- Automata theory is the study of abstract machines (or more appropriately, abstract 'mathematical' machines or systems) and the computational problems that can be solved using these machines. These abstract machines are called automata.
- This automaton consists of
- states (represented in the figure by circles),
- and transitions (represented by arrows).
- Uses of Automata: compiler design and parsing.



WORDS

- Words are strings belonging to some language.
- • Example:
- If Σ= {a} then a language L can be defined as L={an : n=1,2,3,.....} or L={a,aa,aaa,....}

- Here a,aa,… are the words of L All words are strings, but not all strings are words

ALPHABETS AND STRINGS

- A common way to talk about words, number, pairs of words, etc. is by representing them as strings
- To define strings, we start with an alphabet
- An alphabet is a finite set of symbols.
- Example: $\sum 1$= {a, b, c, d, …, z}: the set of letters in English
- $\sum 2$ = {0, 1, …, 9}: the set of (base 10) digits
- $\sum 3$ = {a, b, …, z, #}: the set of letters plus the special symbol #
- $\sum 4$ = {(,)}: the set of open and closed brackets

STRINGS

- A string over alphabet ∑is a finite sequence of symbols in ∑.
- The empty string will be denoted by ∈.
- Examples
- abfbz is a string over ∑1 = {a, b, c, d, …, z}
- 9021 is a string over ∑2 = {0, 1, …, 9}
- ab#bc is a string over ∑3 = {a, b, …, z, #}
-))()(() is a string over S4 = {(,)}

LANGUAGES

- Kinds of languages:

- Talking language
- Programming language
- Formal Languages (Syntactic languages)

LANGUAGES

- A language is a set of strings over an alphabet.
- Languages can be used to describe problems with “yes/no” answers,
- for example:
- L1 = The set of all strings over $\sum 1$ that contain the substring “fool”
- L2 = The set of all strings over $\sum 2$ that are divisible by 7 = {7, 14, 21, …}
- L3 = The set of all strings of the form s#s where s is any string over {a, b, …, z}

REGULAR EXPRESSIONS

- Regular expressions are symbolic notations used to define search patterns in strings. They describe **regular languages** and are commonly used in tasks such as validation, searching, and parsing.
- A **regular expression** represents a **regular language** if it follows these rules:
- **ε (epsilon)** is a Regular Expression indicates the language containing an empty string. **(L (ε) = {ε})**
- **φ** is a Regular Expression denoting an empty language. **(L (φ) = { })**
- **Any symbol 'a' from the input alphabet Σ is a regular expression**, representing the language **{a}**.
- **Union (a + b) is a regular expression** if **a** and **b** are regular expressions, representing the language **{a, b}**.
- **Concatenation (ab) is a regular expression** if **a** and **b** are regular expressions.
- *Kleene star (a) is a regular expression**, meaning **zero or more occurrences of 'a'**, forming a regular language.

REGULAR EXPRESSIONS

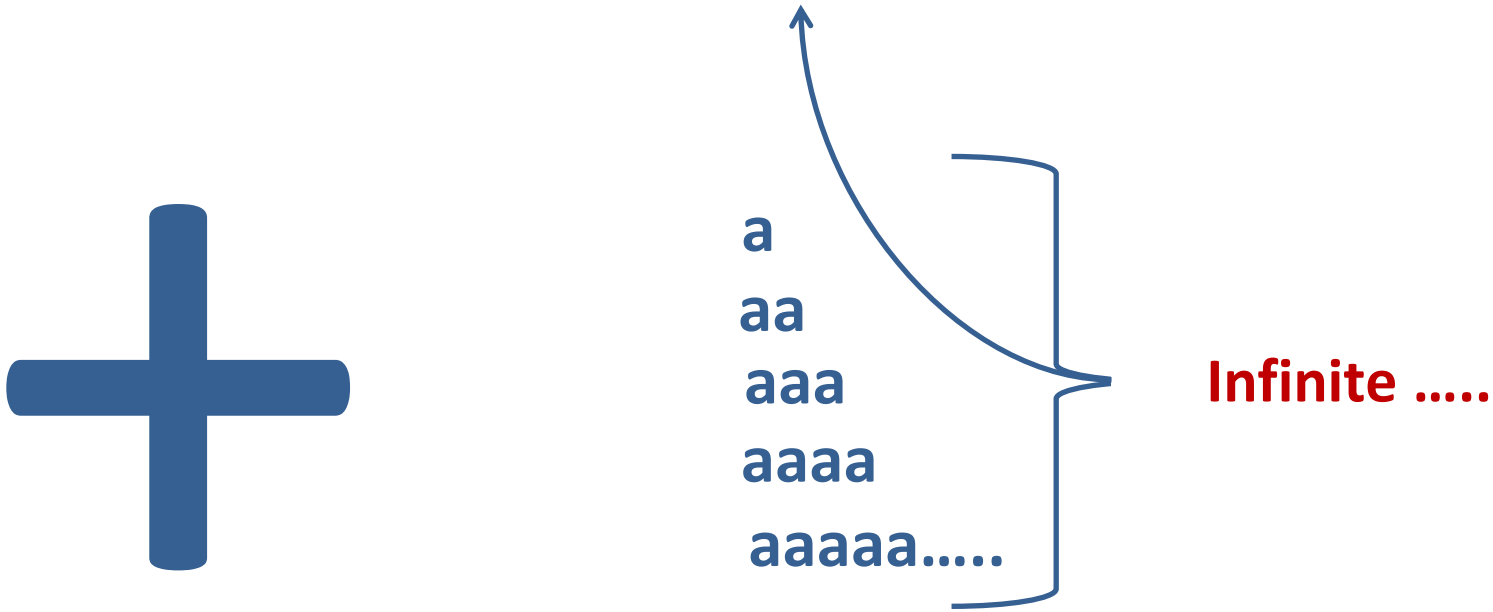
- ✓ A regular expression is a sequence of characters that define a pattern.
- ✓ Application of R.E: Validation, Searching Tools.

❑ Notational shorthand's

1. One or more instances: +
2. Zero or more instances: *
3. Zero or one instances: ?
4. Alphabets: Σ

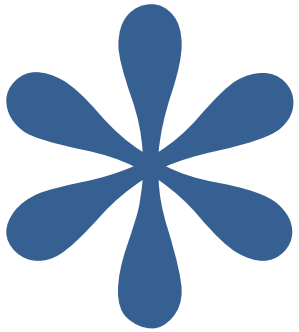
❖ Regular expression

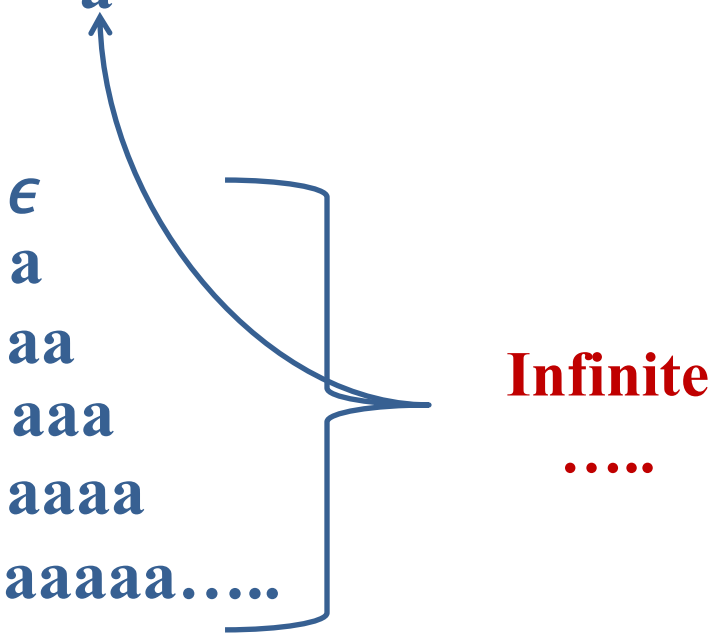
L = One or More Occurrences of a = a^+



❖ Regular expression (Kleene * operator)

L = Zero or More Occurrences of a = a*





❖ Regular expression

1. 0 or 1

   $\boldsymbol{Strings: 0, 1}$ $\qquad$ $\boldsymbol{R.E. = (0 \mid 1)\ also\ written\ as\ (0 + 1)}$

2. 0 or 11 or 111

   $\boldsymbol{Strings: 0, 11, 111}$ $\qquad$ $\boldsymbol{R.E. = (0 \mid 11 \mid 111)\ or\ (0 + 11 + 111)}$

3. String having zero or more a.

   $\boldsymbol{Strings: \epsilon, a, aa, aaa, aaaa \ldots..}$ $\qquad$ $\boldsymbol{R.E. = a^*}$

4. String having one or more a.

   $\boldsymbol{Strings: a, aa, aaa, aaaa \ldots..}$ $\qquad$ $\boldsymbol{R.E. = a^+}$

5. Regular expression over $\Sigma = \{a, b, c\}$ that represent all string of length 3.

   $\boldsymbol{Strings: abc, bca, bbb, cab, aba \ldots.}$ $\qquad$ $\boldsymbol{R.E. = (a + b + c)(a + b + c)(a + b + c)}$

6. All binary string.

   $\boldsymbol{Strings: 0, 1, 11, 00, 101, 10101, 1111 \ldots}$ $\qquad$ $\boldsymbol{R.E = (0 + 1)^+}$

❖ Regular expression

7. 0 or more occurrence of either a or b or both

   $Strings: \epsilon, a, aa, abab, bab \ldots$ $R.E = (a + b)^*$

8. 1 or more occurrence of either a or b or both

   $Strings: a, aa, abab, bab, bbbaaa \ldots$ $R.E. = (a + b)^+$

9. Binary no. ends with 0

   $Strings: 0, 10, 100, 1010, 11110 \ldots$ $R.E = (0 + 1)^* 0$

10. Binary no. ends with 1

    $Strings: 1, 101, 1001, 10101, \ldots$ $R.E = (0 + 1)^* 1$

11. Binary no. starts and ends with 1

    $Strings: 11, 101, 1001, 10101, \ldots$ $R.E = 1(0 + 1)^* 1$

12. String starts and ends with same character

    $Strings: 00, 101, aba, baab \ldots$ $R.E = 1(0 + 1)^* 1$ OR $0\,(0 + 1)^* 0$

    $a(a + b)^* a$ OR $b(a + b)^* b$

❖ Regular expression

13. All string of a and b starting with a

    $Strings: a, ab, aab, abb\ldots$ $R.E = a(a + b)^*$

14. String of 0 and 1 ends with 00

    $Strings: 00, 100, 000, 1000, 1100\ldots$ $R.E = (0 + 1)^*00$

15. String ends with abb

    $Strings: abb, babb, ababb\ldots$ $R.E. = (a + b) * abb$

16. String starts with 1 and ends with 0

    $Strings: 10, 100, 110, 1000, 1100\ldots$ $R.E. = 1(0 + 1)^* 0$

17. All binary string with at least 3 characters and 3rd character should be zero

    $Strings: 000, 100, 1100, 1001\ldots$ $R.E. = (0 + 1)(0 + 1)0(0 + 1)^*$

18. Language which consist of exactly two b's over the set $\Sigma = \{a, b\}$

    $Strings: bb, bab, aabb, abba\ldots$ $R.E. = a^* b a^* b a^*$

❖ Regular expression

19. The language with $\Sigma = \{a, b\}$ such that 3rd character from right end of the string is always a

Strings: *aaa, aba, aaba, abb…* $R.E = (a + b)^*a(a + b)\ (a+ b)$

19. Any no. of a followed by any no. of b followed by any no. of c

Strings: ϵ, *abc, aabbcc, aabc, abb…* $R.E. = a^*\ b^*\ c^*$

20. String should contain at least three 1

Strings: 111, 01101, 0101110…. $R.E. = (0 + 1)^*1\ (0 + 1)^*1\ (0 + 1)^*1\ (0 + 1)^*$

21. String should contain exactly two 1

Strings: 11, 0101, 1100, 010010, 100100…. $R.E. = 0^*10^*10^*$

22. Length of string should be at least 1 and at most 3 over $\Sigma = \{0,1\}$

Strings: 0, 1, 11, 01, 111, 010, 100…. $R.E. = (0 + 1) + (0 + 1)(0 + 1) + (0 + 1)(0 + 1)(0 + 1)$

23. No. of zero should be multiple of 3

Strings: 000, 010101, 110100, 000000, 100010010…. $R.E. = (1^*01^*01^*01^*)^*$

❖ Regular expression

23. The language with $\Sigma = \{a, b, c\}$ where a should be multiple of 3

$\boldsymbol{Strings: aaa, baaa, bacaba, aaaaaa..}$ $\quad \boldsymbol{R.E. = ((b+c)^* a (b+c)^* a (b+c)^* a (b+c)^*)^*}$

24. Even no. of 0

$\boldsymbol{Strings: 00, 0101, 0000, 100100....}$ $\quad \boldsymbol{R.E. = (1^*01^*01^*)^*}$

25. String should have odd length

$\boldsymbol{Strings: 0, 010, 110, 000, 10010....}$ $\quad \boldsymbol{R.E. = (0+1)\ ((0+1)\ (0+1))^*}$

26. String should have even length

$\boldsymbol{Strings: 00, 0101, 0000, 100100....}$ $\quad \boldsymbol{R.E. = ((0+1)\ (0+1))^*}$

27. String start with 0 and has odd length

$\boldsymbol{Strings: 0, 010, 010, 000, 00010....}$ $\quad \boldsymbol{R.E. = 0\ ((0+1)(0+1))^*}$

❖ Regular expression

29. All string begins or ends with 00 or 11

$Strings: 00101, 10100, 110, 01011 \ldots$ $R.E = (00 + 11)\,(0 + 1)^* + (0 + 1)^*\,(00 + 11)$

30. Language of all string containing both 11 and 00 as substring

$Strings: 0011, 1100, 100110, 010011 \ldots$

$R.E. = ((0 + 1)^*00(0 + 1)^*11(0 + 1)^*) + ((0 + 1)^*11(0 + 1)^*00(0 + 1)^*)$

31. String ending with 1 and not contain 00

$Strings: 011, 1101, 1011 \ldots.$ $R.E = (1 + 01)^+$

REGULAR GRAMMAR

- Grammars denote syntactical rules for conversation in natural languages.
- **Representation of Grammar**

A grammar **G** can be formally written as a 4-tuple (N, T, S, P) where –

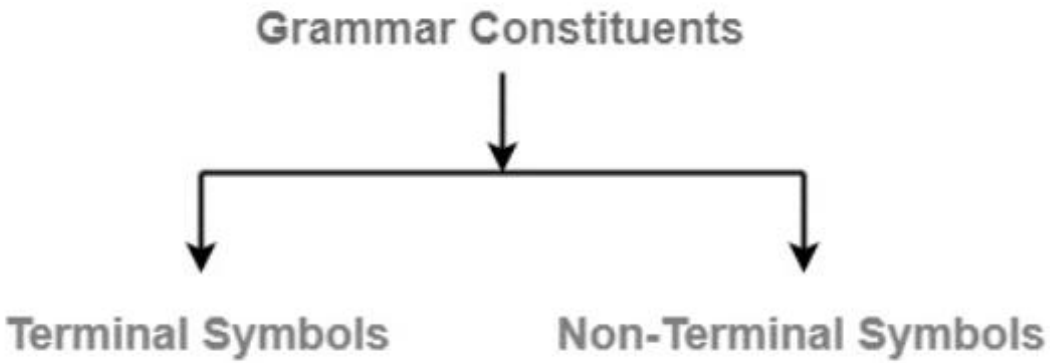
- N or VN is a set of variables or non-terminal symbols.
- **T** or ∑ is a set of Terminal symbols.
- **S** is a special variable called the Start symbol, S ∈ N
- P is Production rules for Terminals and Non-terminals. A production rule has the form α → β, where α and β are strings on VN ∪ ∑ and least one symbol of α belongs to VN.

REGULAR GRAMMAR

- A regular grammar is a formal grammar that generates regular languages. It consists of:
- **Terminals:** Symbols that form strings (e.g., a, b).
- **Non-terminals:** Variables used to derive strings (e.g., S, A).
- **Production Rules:** Rules for transforming non-terminals into terminals or other non-terminals.
- **Start Symbol:** The non-terminal from which derivations begin

BASIC ELEMENTS OF GRAMMAR

- Grammar is composed of two basic elements



BASIC ELEMENTS OF GRAMMAR

- **Terminal Symbols** - Terminal symbols are the components of the sentences that are generated using grammar and are denoted using small case letters like a, b, c etc.
- **Non-Terminal Symbols** - Non-Terminal Symbols take part in the generation of the sentence but are not the component of the sentence. These types of symbols are also called Auxiliary Symbols and Variables. They are represented using a capital letter like A, B, C, etc.

REGULAR GRAMMAR

- **Example 1**

Grammar G1 – ({S, A, B}, {a, b}, S, {S → AB, A → a, B → b})

- **S, A,** and **B** are Non-terminal symbols;
- **a** and **b** are Terminal symbols
- **S** is the Start symbol, S ∈ N
- Productions, **P : S → AB, A → a, B → b**

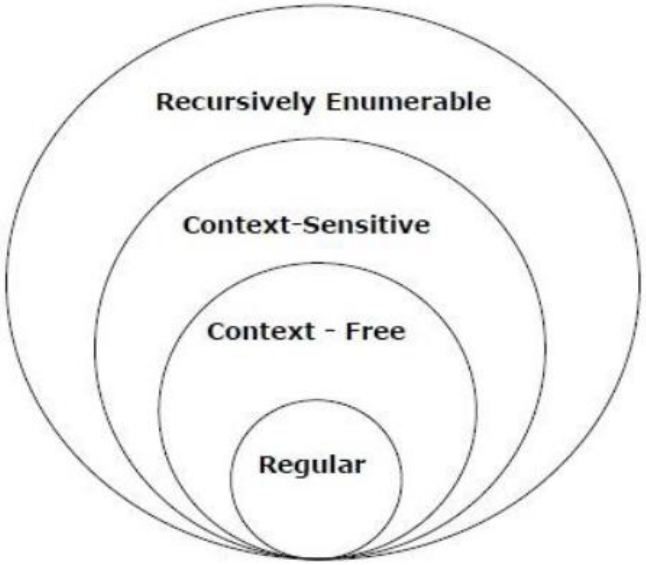
REGULAR GRAMMAR

- **Example 2**
- Grammar G2 –(({S, A}, {a, b}, S,{S → aAb, aA → aaAb, A → ε })

- **S** and **A** are Non-terminal symbols.
- **a** and **b** are Terminal symbols.
- **ε** is an empty string.
- **S** is the Start symbol, S ∈ N
- Production **P : S → aAb, aA → aaAb, A → ε**

TYPES OF GRAMMAR

Grammar	Language	Automata	Production rules
Type-0	Recursively enumerable	Turing machine	No restriction
Type-1	Context-sensitive	Linear-bounded non-deterministic machine	$\alpha A\beta \rightarrow \alpha\gamma\beta$
Type-2	Context-free	Non-deterministic push down automata	$A \rightarrow \gamma$
Type-3	Regular	Finite state automata	$A \rightarrow \alpha B$ $A \rightarrow \alpha$



FINITE AUTOMATA

- A finite automata is an abstract machine used to model computation. It consists of a finite number of states and operates on input symbols to transition between these states based on a set of rules.
- Basic model of finite automata consists of :

(i) An input tape divided into cells, each cell can hold a symbol

(ii) A read head which can read one symbol at a time from a finite alphabet

(iii) A finite control which works within a finite set of states. At each step, it changes its state depending on the current state and input read. Its change of state is specified by a transition function. It accepts the input if it is in a set of accepting states.

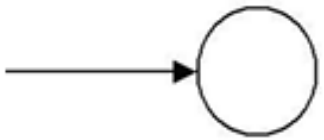
FINITE AUTOMATA

- A finite automata, or finite state machine is a 5-tuple (Q,Σ,δ,q0,F) where,
- **Q** – A finite, non-empty set of states.
- **Σ** – A finite, non-empty set of input alphabets (symbols).
- **δ** – The transition function, which maps into .
- **q0** – The initial state, which is an element of .
- **F** – A subset of representing the set of final states.

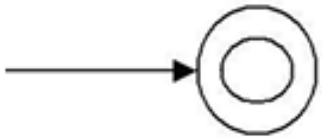
TRANSITION DIAGRAMS AND TRANSITION SYSTEMS

- A transition graph or a transition system is a finite directed labeled graph in which each vertex (node) represents a state and the directed edges indicate the transition of a state and the edges are labelled with input.
- A transition graph contains:

(i) A finite set of states, one of which are designated as start state and some of which are designated as final states.



Start state



Final state

CONT..

- (ii) An alphabet of possible input letters from which input strings are formed.
- (iii) A finite set of transitions that show, how to go from some states to some other states. So a transition system is a 5-tuple $(Q,\sum,\delta,q0,F)$

FINITE AUTOMATA MODEL:

- Finite automata can be represented by input tape and finite control.
- **Input tape:** It is a linear tape having some number of cells. Each input symbol is placed in each cell.
- **Finite control:** The finite control decides the next state on receiving particular input from input tape. The tape reader reads the cells one by one from left to right, and at a time only one input symbol is read.

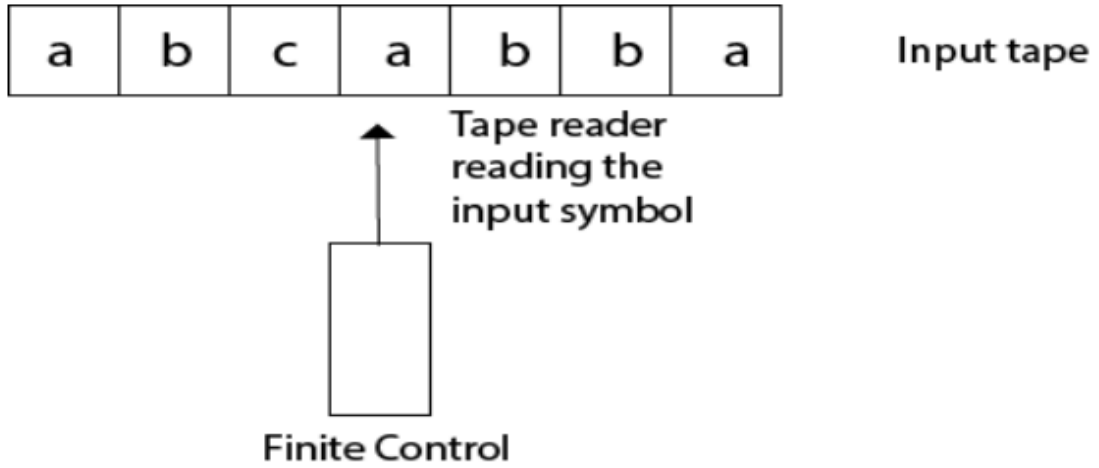


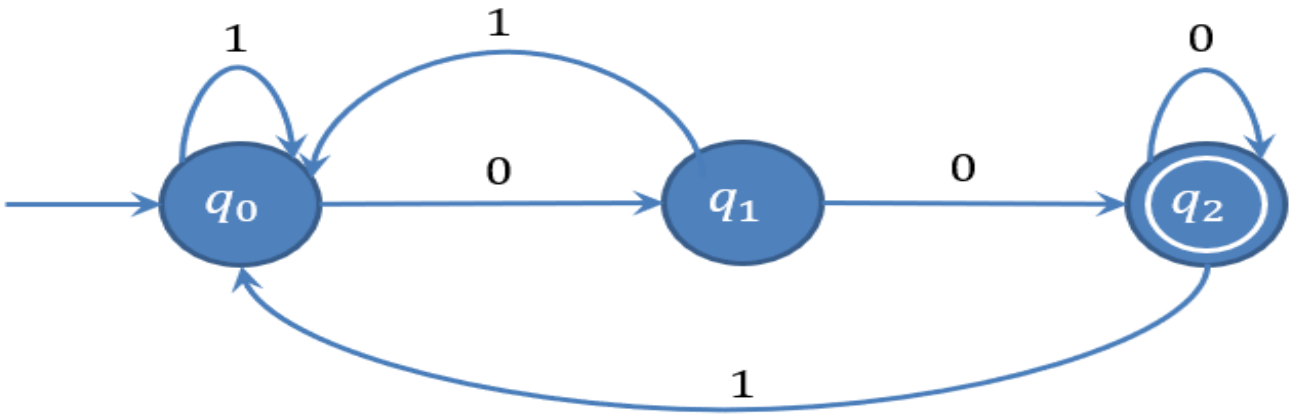
Fig :- Finite automata model

Example: Finite Automata

- $M = (Q, \Sigma, q_0, A, \delta)$
  - $Q = \{q_0, q_1, q_2\}$
  - $\Sigma = \{0,1\}$
  - $q_0 = q_0$
  - $A = \{q_2\}$
  - δ is defined as

Transition Table

δ	Input	
State	0	1
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_0

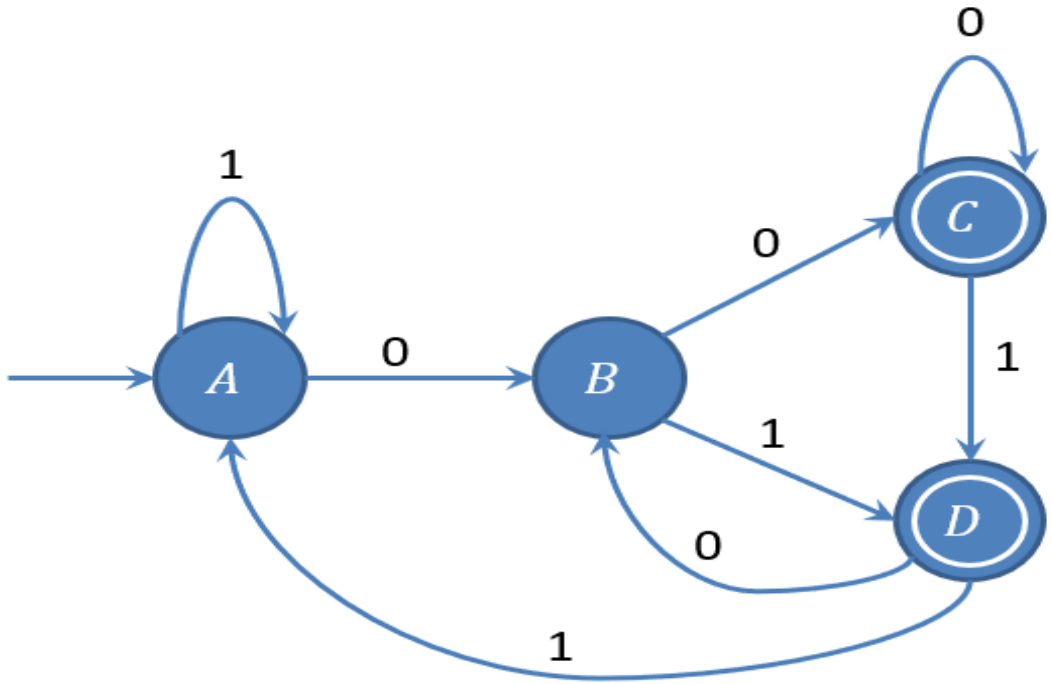


APPLICATION OF FA

- Lexical analysis phase of a compiler
- Design of digital circuit
- String matching
- Communication protocol for information exchange.

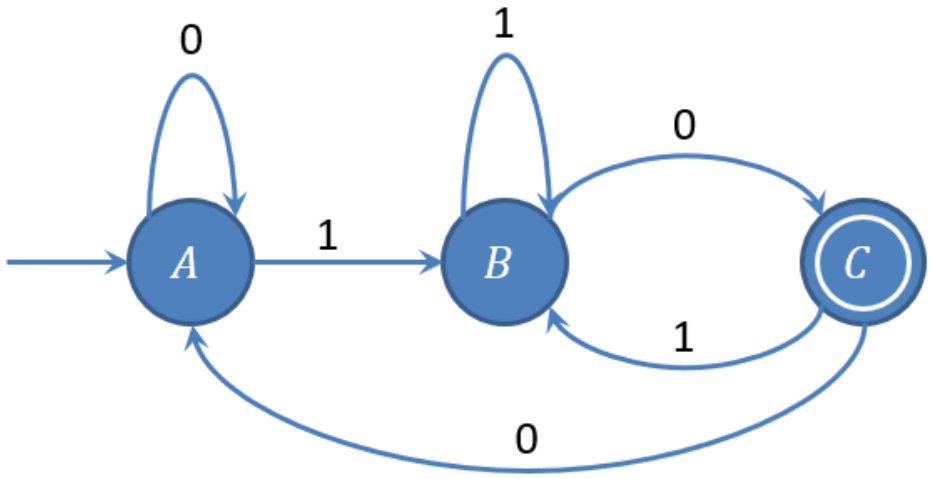
FA EXAMPLES

- The string with next to last symbol as 0.



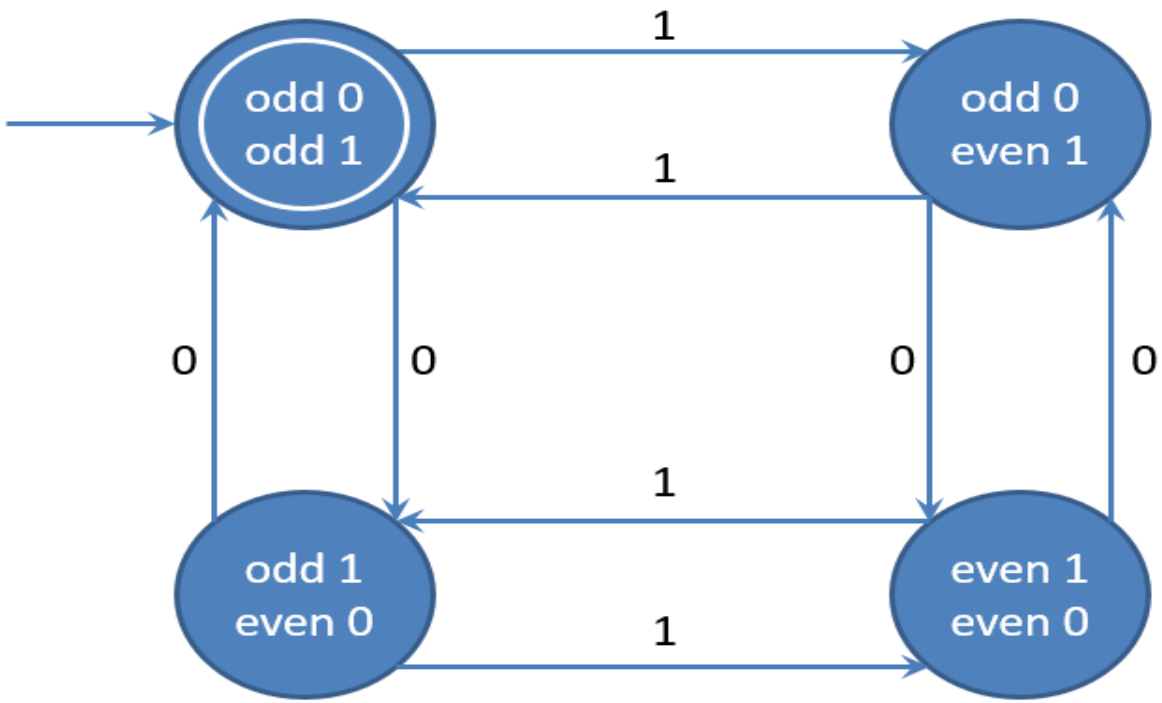
FA EXAMPLES

- The strings ending with 10.



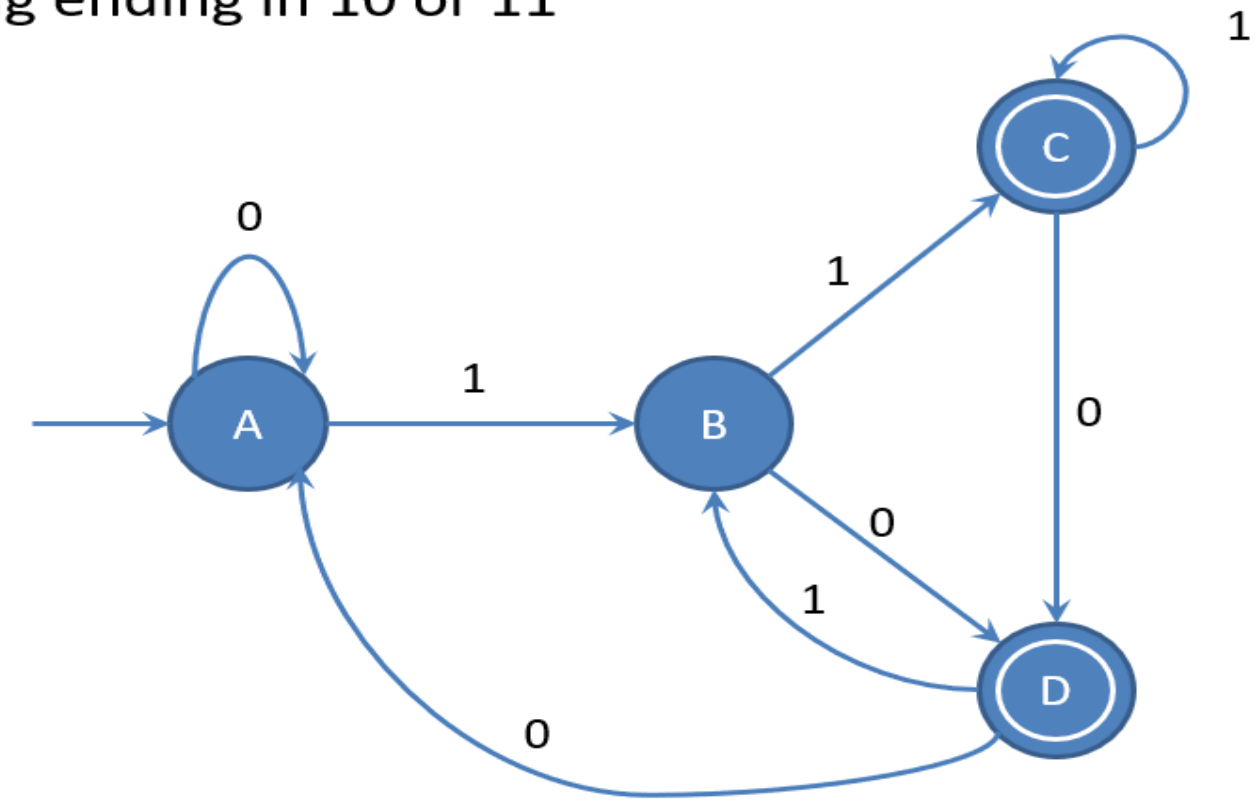
FA EXAMPLES

- The string with number of 0's and number of 1's are odd



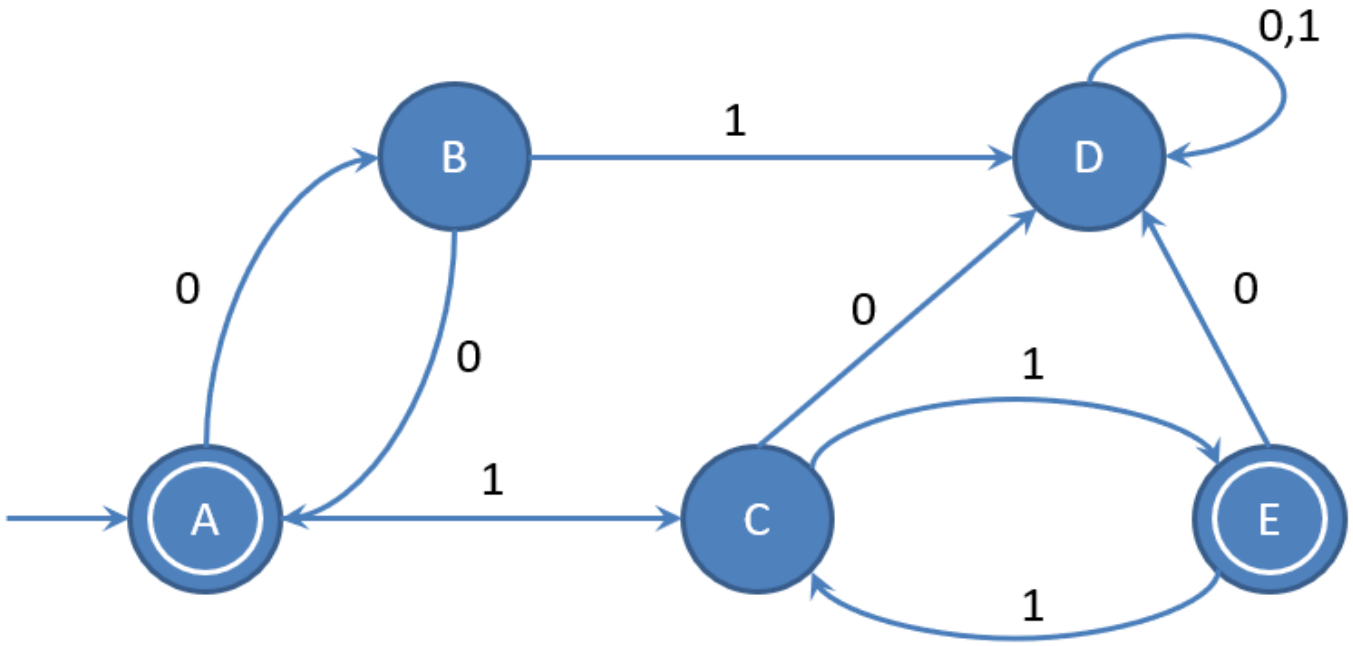
FA EXAMPLES

- The string ending in 10 or 11



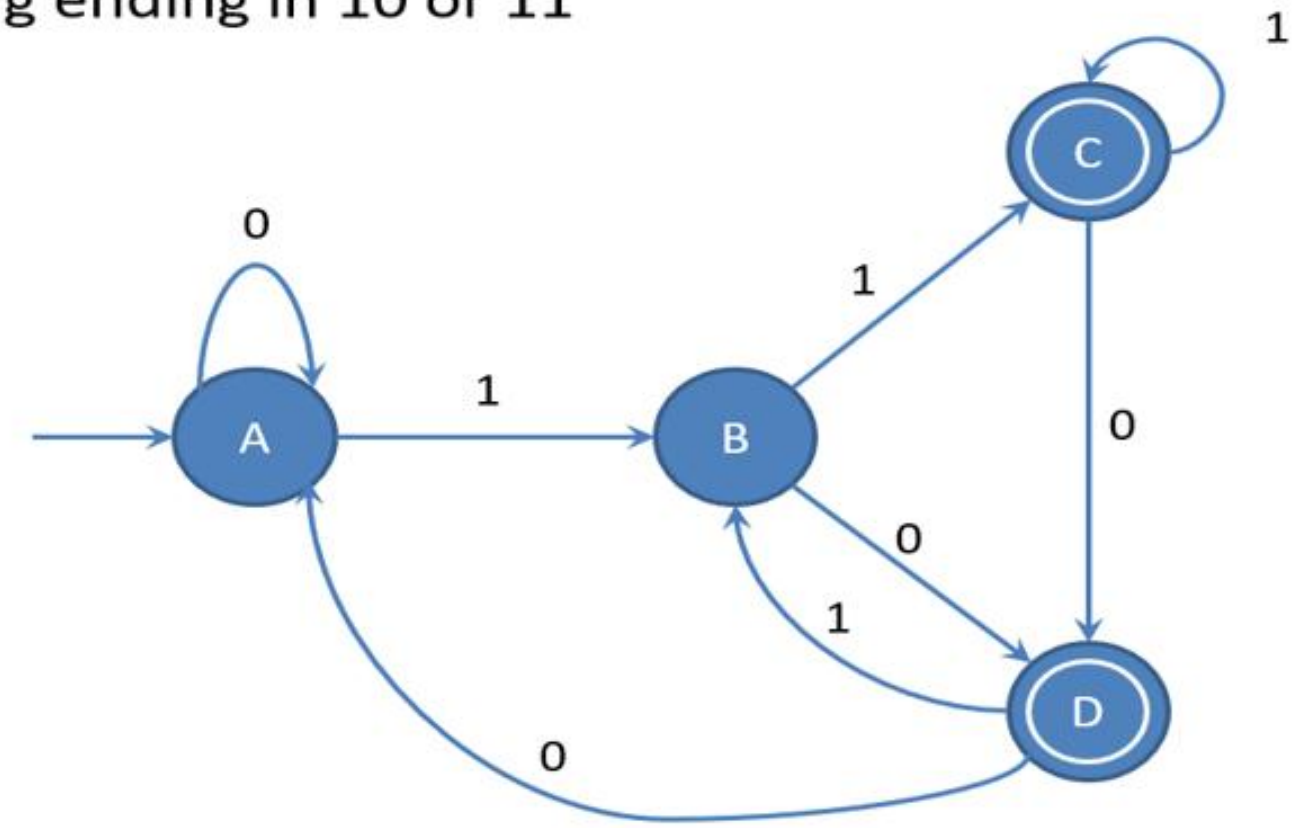
FA EXAMPLES

- The string corresponding to Regular expression {00}*{11}*



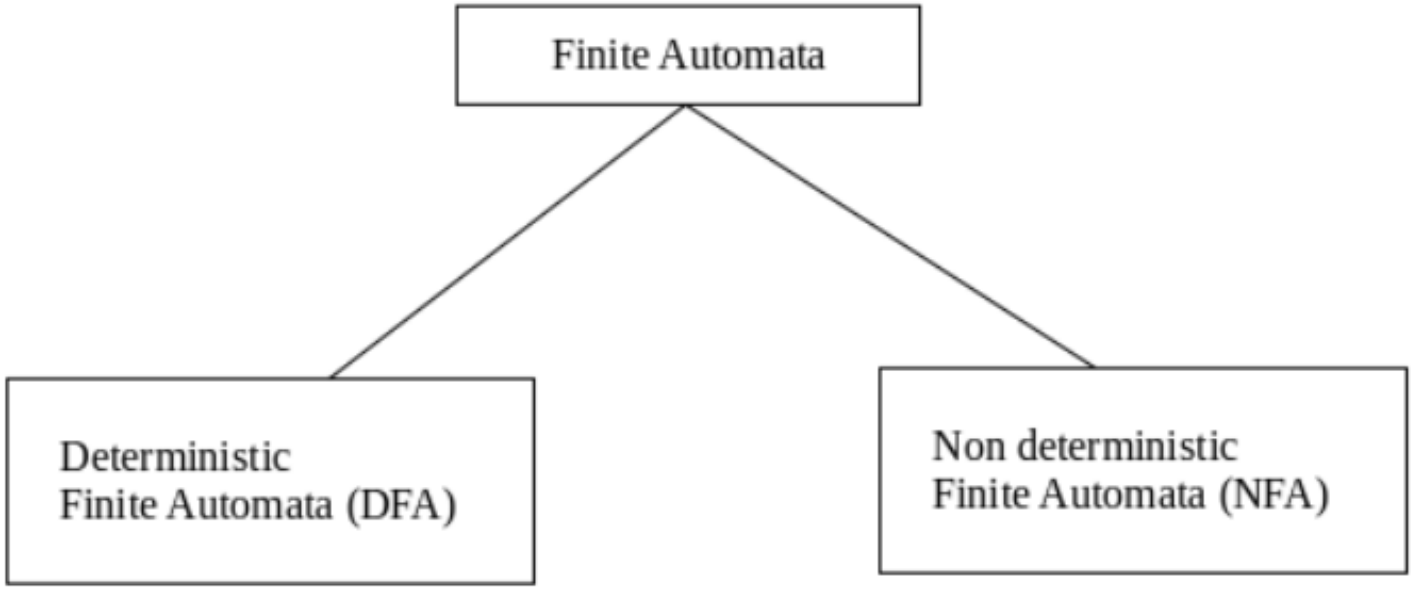
FA EXAMPLES

- The string ending in 10 or 11



TYPES OF AUTOMATA:

- There are two types of finite automata:
- DFA(deterministic finite automata)
- NFA(non-deterministic finite automata)



DFA (DETERMINISTIC FINITE AUTOMATA)

- DFA refers to deterministic finite automata. Deterministic refers to the uniqueness of the computation. The finite automata are called deterministic finite automata if the machine is read an input string one symbol at a time.
- In DFA, there is only one path for specific input from the current state to the next state.
- DFA does not accept the null move, i.e., the DFA cannot change state without any input character.
- DFA can contain multiple final states. It is used in Lexical Analysis in Compiler.

DFA

- In the following diagram, we can see that from state q0 for input a, there is only one path which is going to q1. Similarly, from q0, there is only one path for input b going to q2.

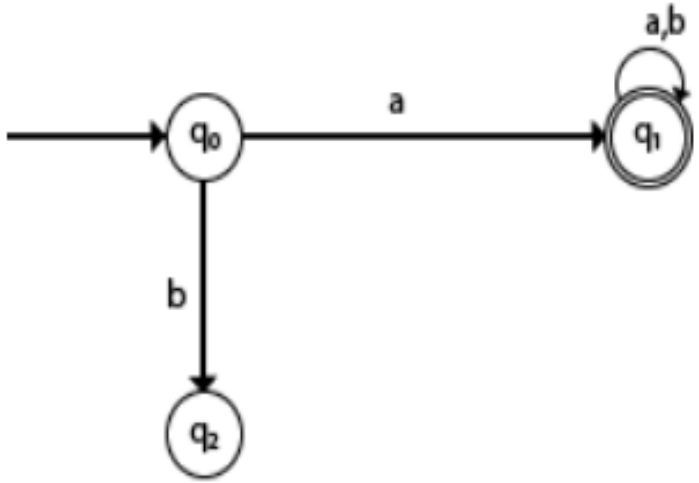


Fig:- DFA

FORMAL DEFINITION OF DFA

- A DFA is a collection of 5-tuples same as we described in the definition of FA. (Q, ∑, δ, q0, F)
- Q: finite set of states
- ∑: finite set of the input symbol
- q0: initial state
- F: **final** state
- δ: Transition function
- **Transition function can be defined as:**

  $\delta: Q \times \sum \rightarrow Q$

GRAPHICAL REPRESENTATION OF DFA

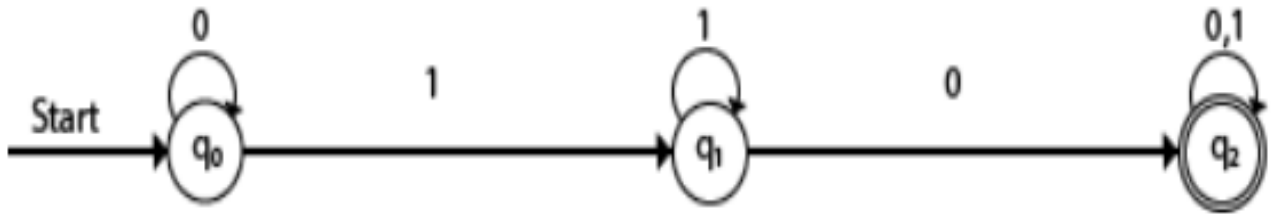
- A DFA can be represented by digraphs called state diagram. In which:
- The state is represented by vertices.
- The arc labeled with an input character show the transitions.
- The initial state is marked with an arrow.
- The final state is denoted by a double circle.

EXAMPLE 1:

- Q = {q0, q1, q2}
- ∑ = {0, 1}
- q0 = {q0}
- F = {q2}
- **Solution:**
- Transition Diagram:

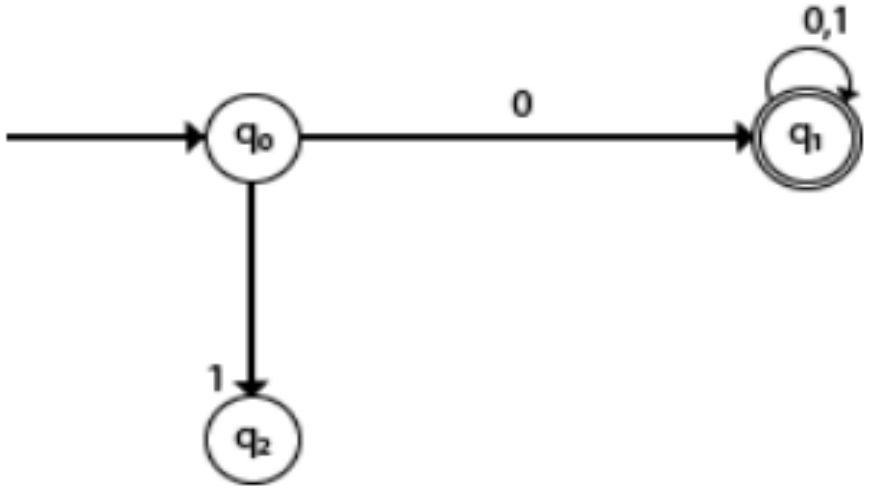
Transition Table:

Present State	Next state for Input 0	Next State of Input 1
→q0	q0	q1
q1	q2	q1
*q2	q2	q2



EXAMPLE 2:

- DFA with ∑ = {0, 1} accepts all starting with 0.
  Solution:

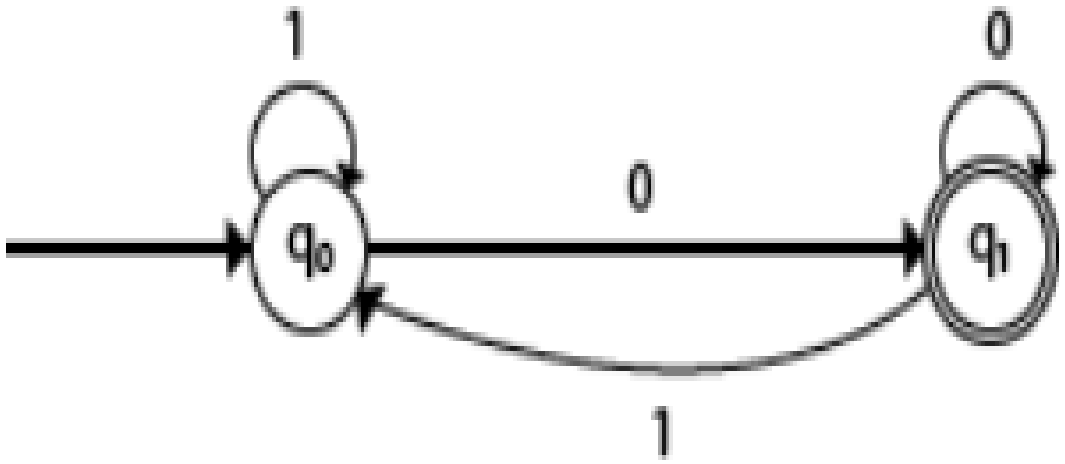


Explanation:

•In the above diagram, we can see that on given 0 as input to DFA in state q0 the DFA changes state to q1 and always go to final state q1 on starting input 0. It can accept 00, 01, 000, 001....etc. It can't accept any string which starts with 1, because it will never go to final state on a string starting with 1.

EXAMPLE 3:

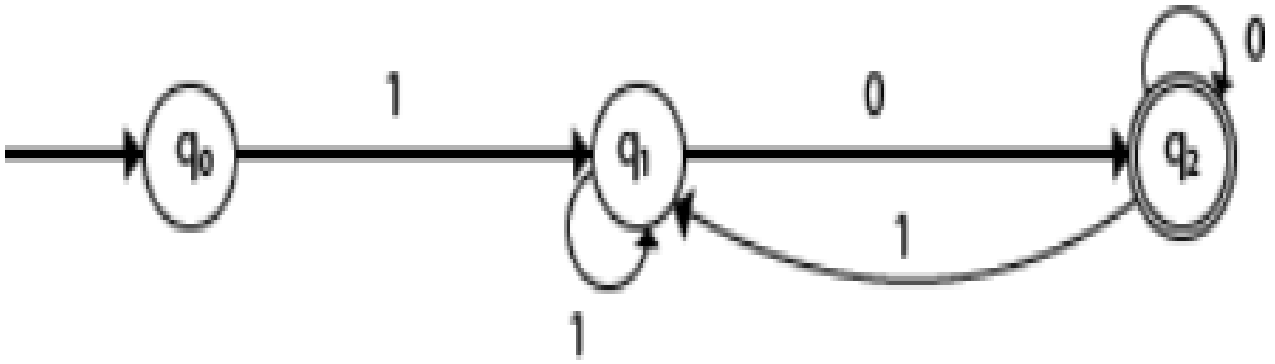
- DFA with ∑ = {0, 1} accepts all ending with 0.
- **Solution:**



Explanation:
In the above diagram, we can see that on given 0 as input to DFA in state q0, the DFA changes state to q1. It can accept any string which ends with 0 like 00, 10, 110, 100....etc. It can't accept any string which ends with 1, because it will never go to the final state q1 on 1 input, so the string ending with 1, will not be accepted or will be rejected.

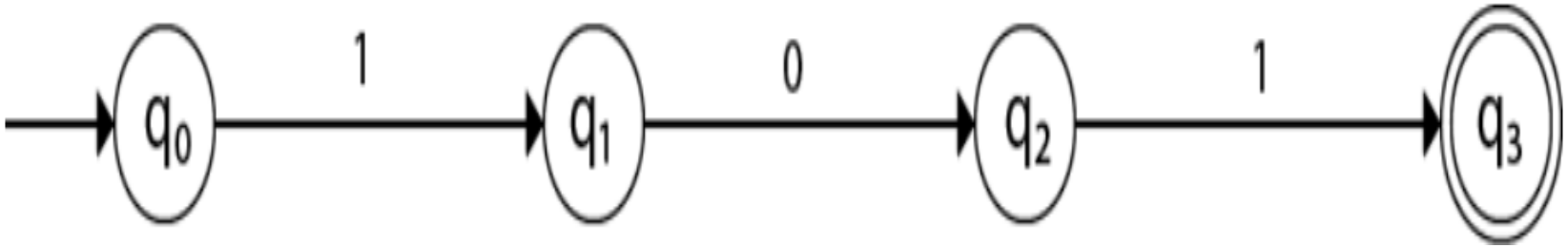
EXAMPLES OF DFA

- **Example 1:**
- Design a FA with ∑ = {0, 1} accepts those string which starts with 1 and ends with 0.
- **Solution:**
- The FA will have a start state q0 from which only the edge with input 1 will go to the next state.



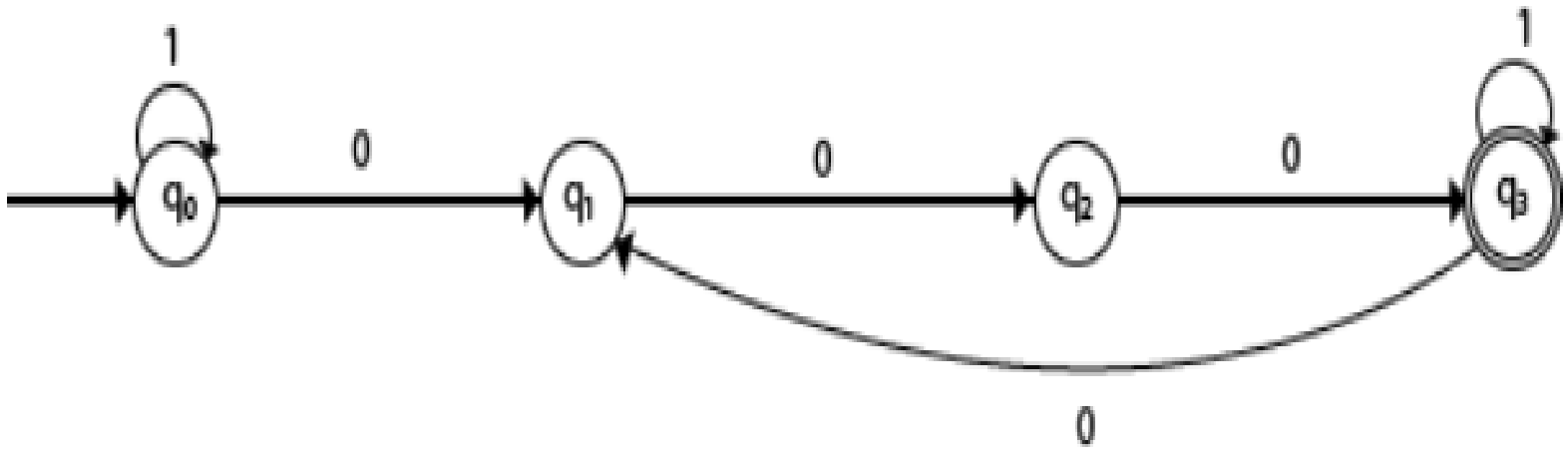
EXAMPLE 2:

- Design a FA with $\sum = \{0, 1\}$ accepts the only input 101.



EXAMPLE 3

- Design FA with ∑ = {0, 1} accepts the set of all strings with three consecutive 0's.
- The strings that will be generated for this particular languages are 000, 0001, 1000, 10001, ... in which 0 always appears in a clump of 3. The transition graph is as follows:



NFA (NON-DETERMINISTIC FINITE AUTOMATON)

- A **Nondeterministic Finite Automaton (NFA)** is a type of finite automaton where multiple transitions are allowed for a given state and input symbol.
- In some cases, there may even be transitions without any input symbol (ε-transitions).
- NFAs are flexible and easier to construct than their deterministic counterparts, though they are computationally equivalent to DFAs.

NFA

- **Key Features of NFA**
- NFAs have the following characteristics:
- **Multiple Transitions:** For a given state and input symbol, an NFA can transition to multiple next states.
- **ε-Transitions:** An NFA can transition from one state to another without consuming any input symbol.
- **Acceptance:** An NFA accepts a string if there exists at least one sequence of transitions that ends in an accept state after processing the input.
- **Non-Determinism:** Unlike DFAs, the behavior of an NFA is not predictable at each step as it may have multiple choices for transitions.

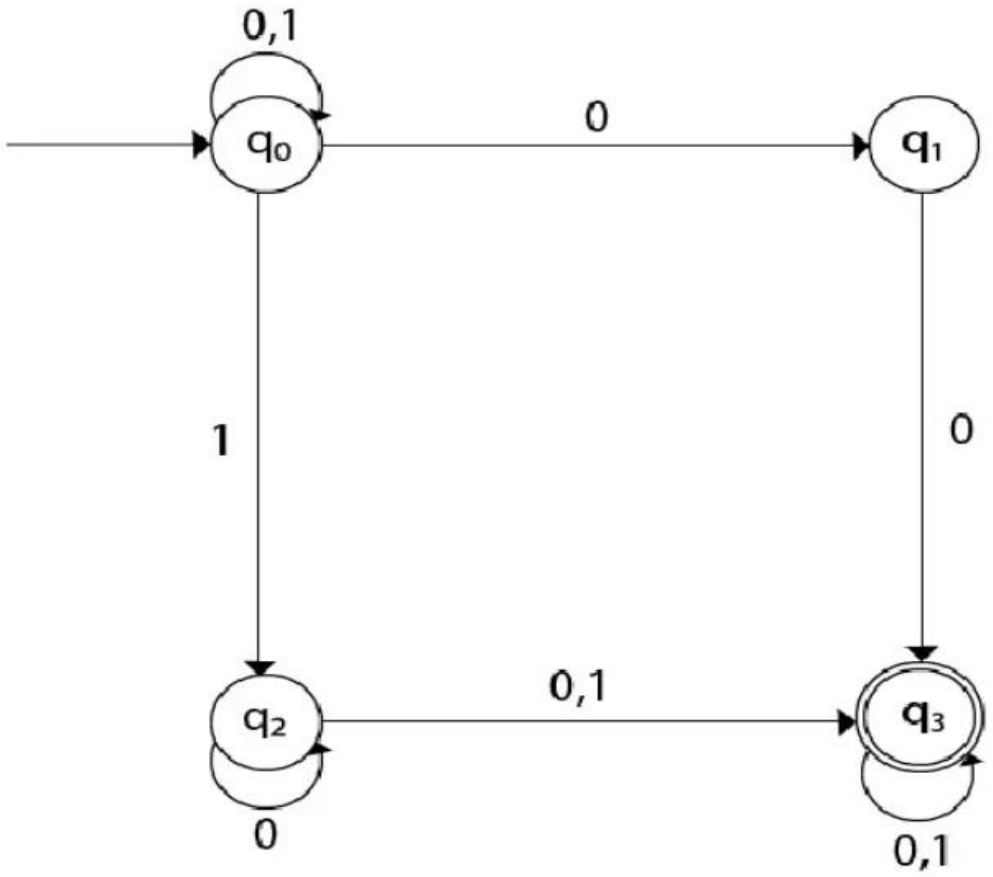
FORMAL DEFINITION OF NFA

- An NFA is formally defined as a 5-tuple **$(Q, \Sigma, \delta, q_0, F)$**, where:
- **Q:** A finite set of states.
- **Σ:** A finite input alphabet.
- **δ:** A transition function, **$\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^Q$**, which maps a state and input symbol (or ε) to a set of next states.
- **q_0:** The start state, where the NFA begins processing.
- **F:** A set of accept states, $F \subseteq Q$.

EXAMPLE 1

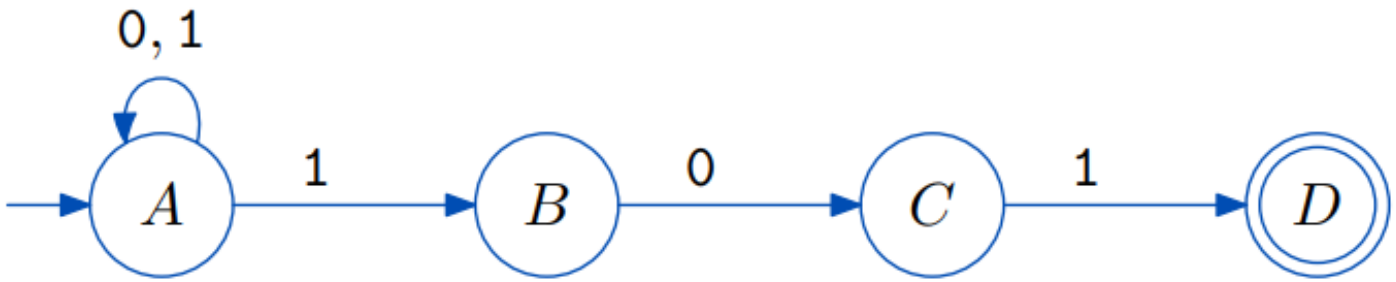
Design a NFA for the transition table as given below:

Present State	0	1
→q0	q0, q1	q0, q2
q1	q3	ε
q2	q2, q3	q3
→q3	q3	q3



EXAMPLE 2

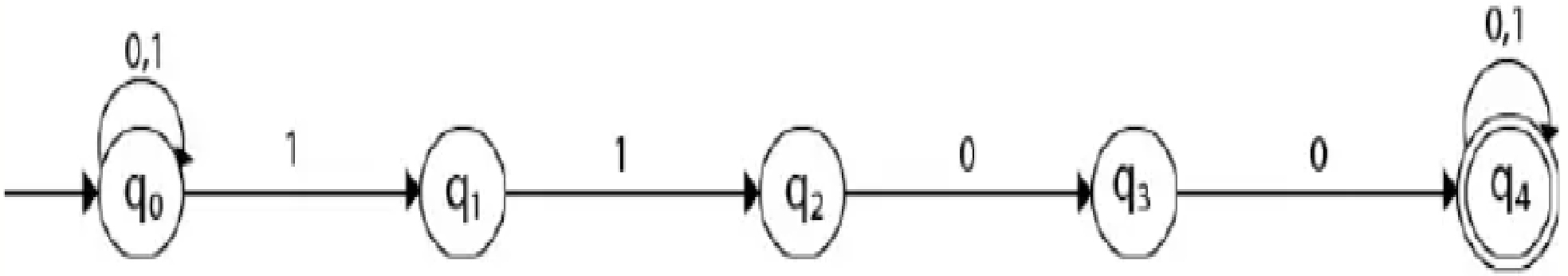
- An NFA that accepts all binary strings that end with 101



EXAMPLE 3

Design an NFA with Σ = {0, 1} in which double '1' is followed by double '0'.

L={1100,011000,111001,011001,111000.....}

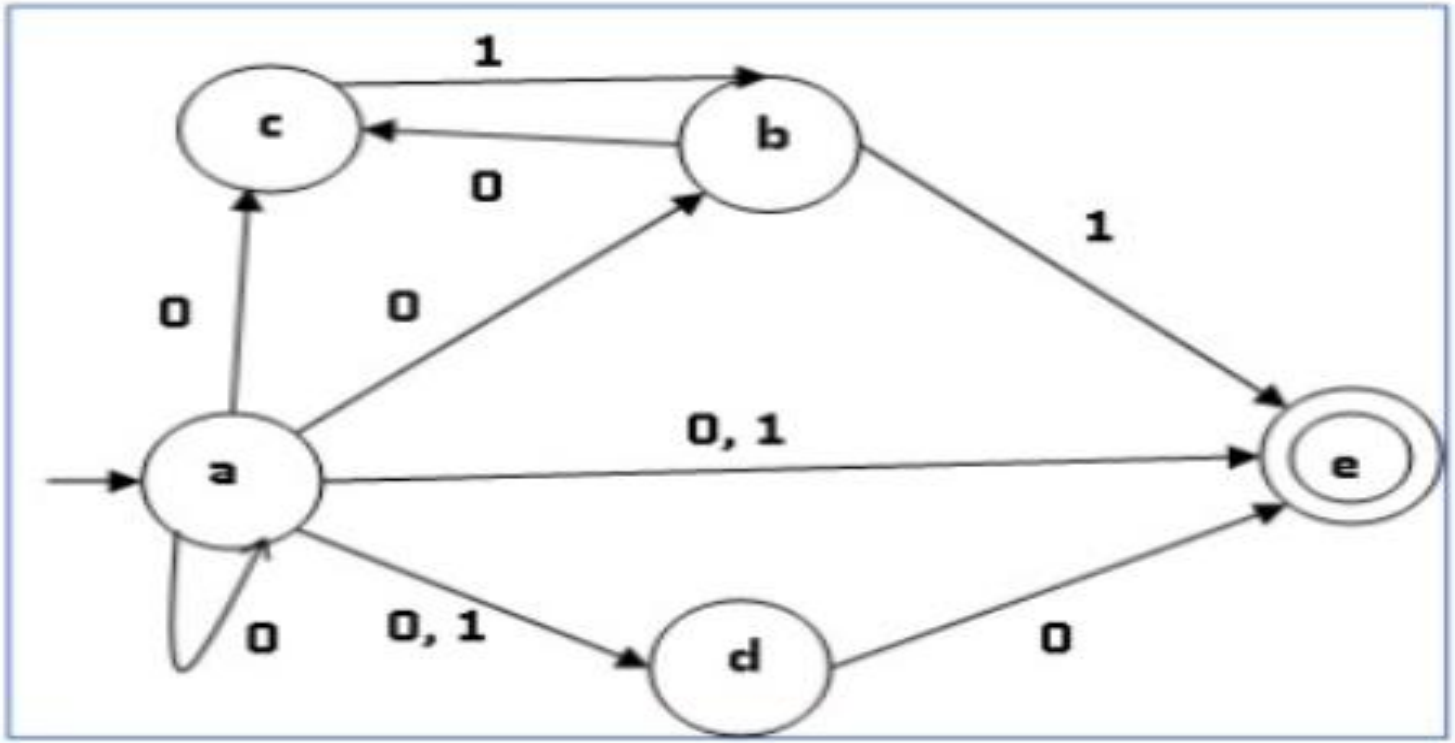


CONVERSION OF NFA TO DFA

- Let $\mathbf{X = (Q_x, \sum, \delta_x, q_0, F_x)}$ be an NDFA which accepts the language L(X). We have to design an equivalent DFA $\mathbf{Y = (Q_y, \sum, \delta_y, q_0, F_y)}$ such that **L(Y) = L(X)**. The following procedure converts the NDFA to its equivalent DFA –
- **Algorithm**
- **Input** – An NDFA
- **Output** – An equivalent DFA
- **Step 1** – Create state table from the given NDFA.
- **Step 2** – Create a blank state table under possible input alphabets for the equivalent DFA.
- **Step 3** – Mark the start state of the DFA by q0 (Same as the NDFA).
- **Step 4** – Find out the combination of States $\{Q_0, Q_1, \ldots, Q_n\}$ for each possible input alphabet.
- **Step 5** – Each time we generate a new DFA state under the input alphabet columns, we have to apply step 4 again, otherwise go to step 6.
- **Step 6** – The states which contain any of the final states of the NDFA are the final states of the equivalent DFA.

EXAMPLE

LET US CONSIDER THE NDFA SHOWN IN THE FIGURE BELOW.

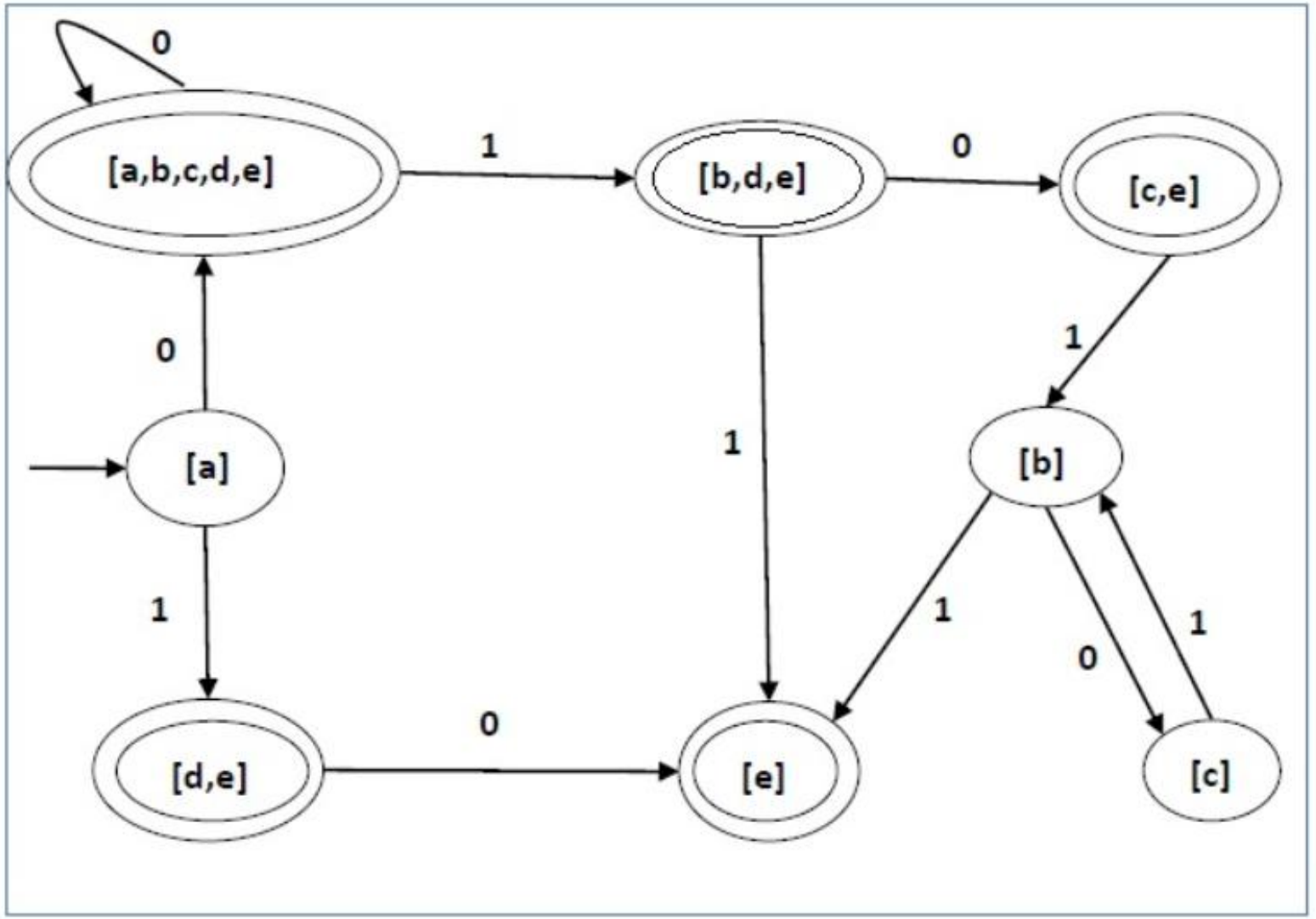


q	(q,0)	(q,1)
a	{a,b,c,d,e}	{d,e}
b	{c}	{e}
c	∅	{b}
d	{e}	∅
e	∅	∅

Using the above algorithm, we find its equivalent DFA. The state table of the DFA is shown in below.

q	δ(q,0)	δ(q,1)
[a]	[a,b,c,d,e]	[d,e]
[a,b,c,d,e]	[a,b,c,d,e]	[b,d,e]
[d,e]	[e]	∅
[b,d,e]	[c,e]	[e]
[e]	∅	∅
[c, e]	∅	[b]
[b]	[c]	[e]
[c]	∅	[b]

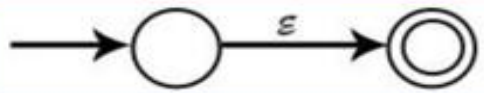
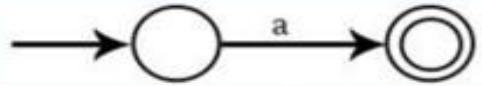
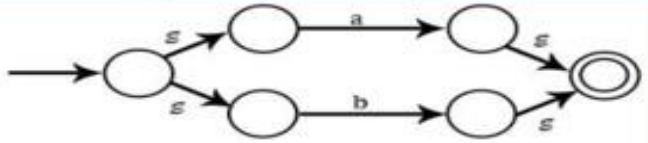
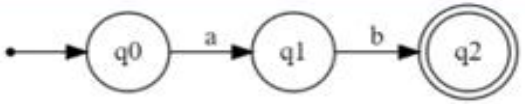
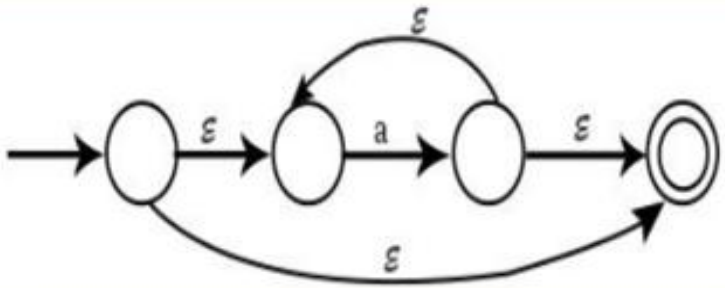
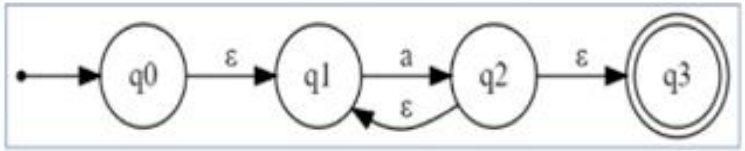
The state diagram of the DFA is as follows –



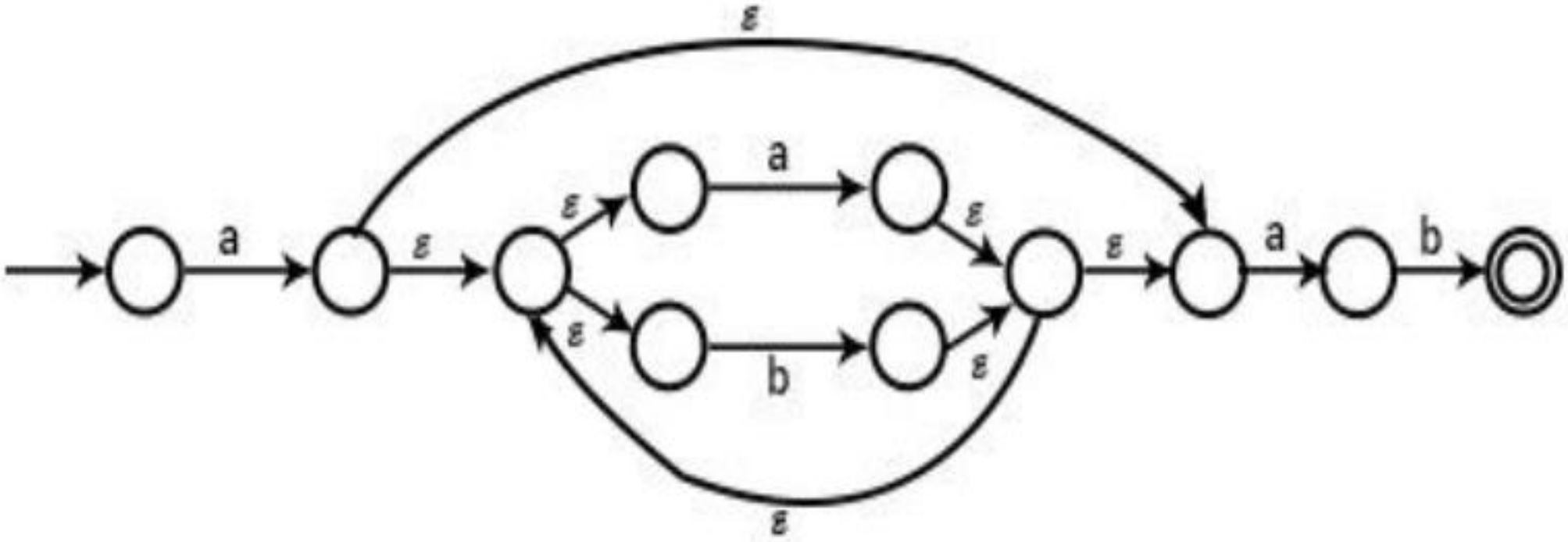
Conversion of Regular Expression to NF

REGULAR EXPRESSION TO ∈-NFA

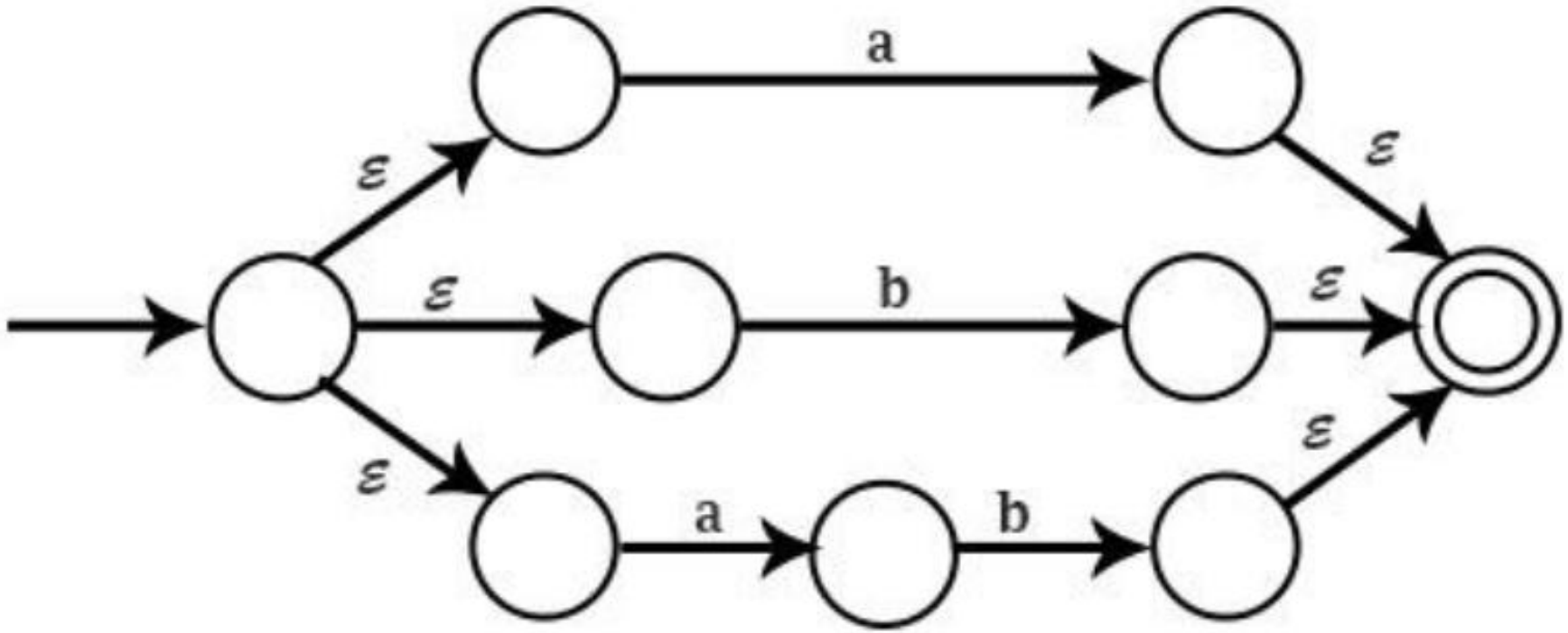
- An ∈ - NFA is similar to an NFA but with a key difference: it allows transitions on the empty string (denoted by ∈).
- This means that the automaton can change its state without consuming any input symbols.
- Algorithm for the conversion of Regular Expression to NFA as follows:

RE	€-NFA
ε	
a	
a + b, or a | b	
ab	 
a*	
a+	

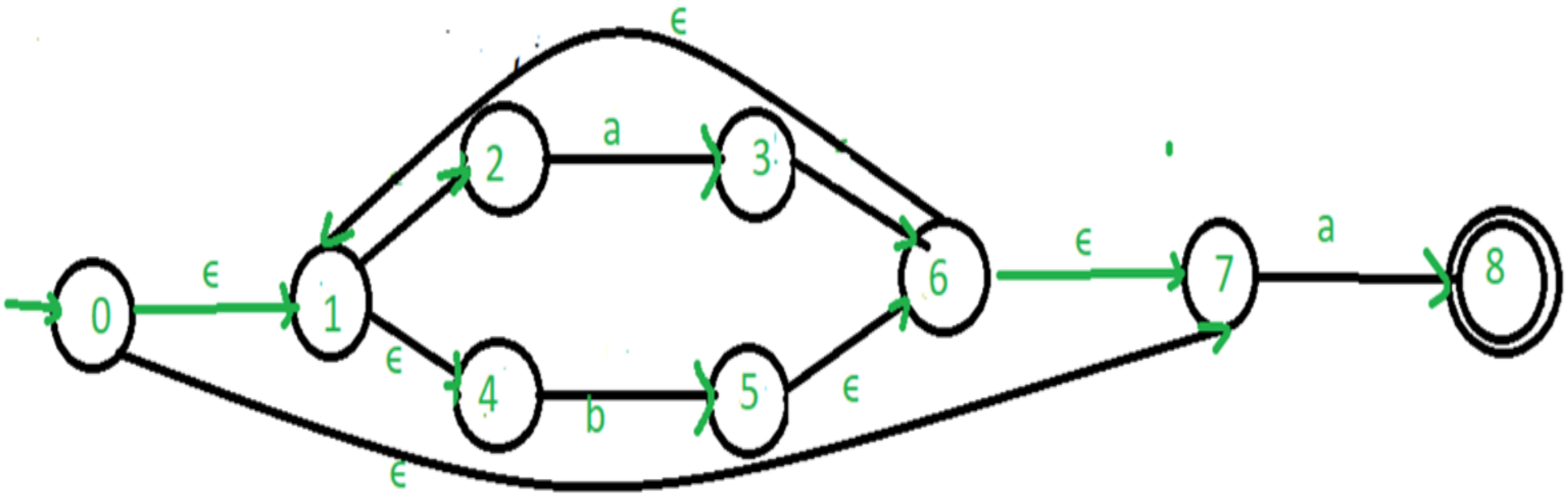
EXAMPLE 1: DRAW NFA FOR THE REGULAR EXPRESSION a(a+b)*ab



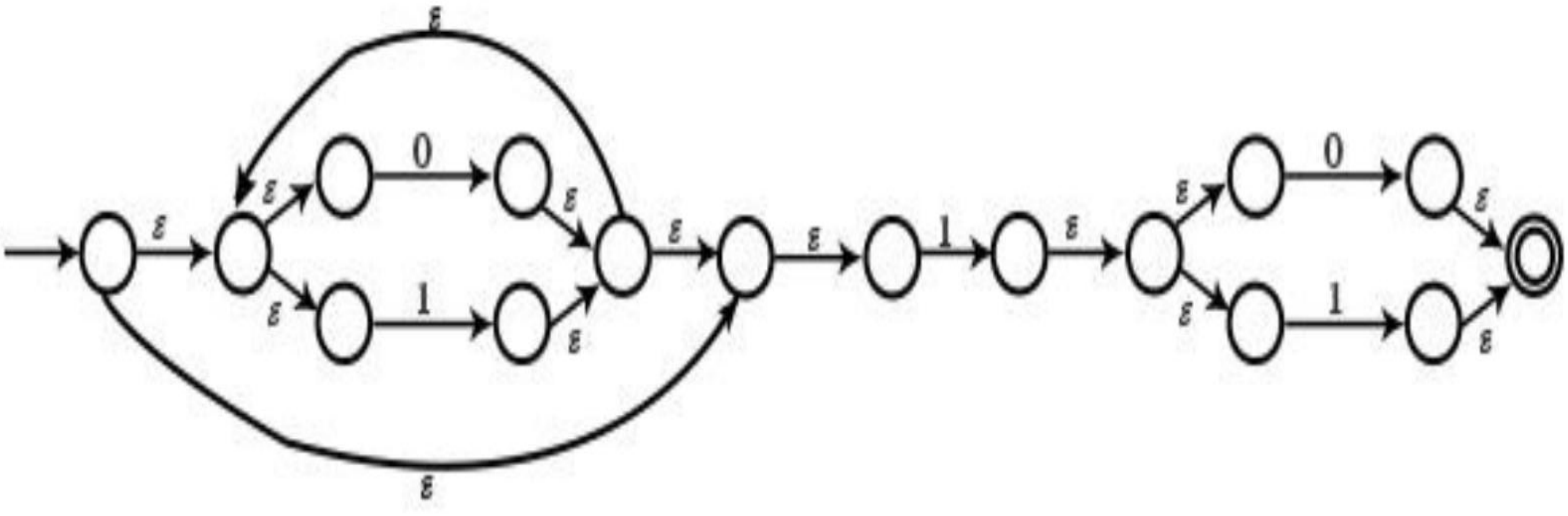
EXAMPLE 2: DRAW NFA FOR a + b + ab



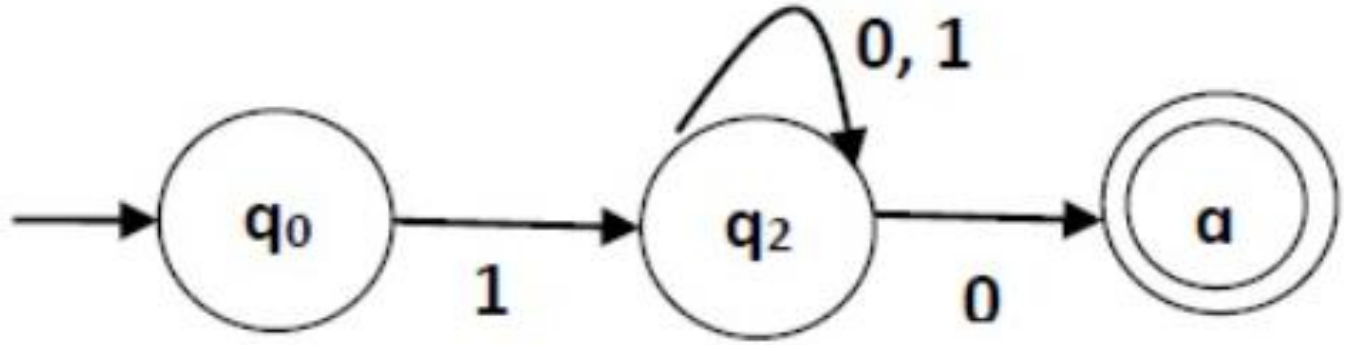
EXAMPLE 3: CREATE A ∈-NFA FOR REGULAR EXPRESSION: (a/b)*a



EXAMPLE 4: DRAW NFA CORRESPONDING TO (0+1)*1(0+1)



EXAMPLE 5: CONVERT THE FOLLOWING RA INTO ITS EQUIVALENT 1 (0 + 1)* 0



THANK YOU